SPÉCIFICATIONS FONCTIONNELLES & TECHNIQUES

CAHIER DES CHARGES — APPLICATION DESKTOP CROSS-PLATFORM

Projet : OptiMedia Desktop (Images & Vidéos) **Stack :** Tauri v2 · React.js · Rust

Cibles : Windows (.exe, .msi) · Linux (.deb) · macOS (.dmg) **Version :** v1.1 — Spécifications Médias

Règles Métier & Quotas de Traitement Multimédia

• **Compression d'Images :** Import d'images haute définition d'une taille moyenne de **15 Mo** (jusqu'à 50 fichiers/lot) pour une taille en sortie calibrée entre **800 Ko et 2 Mo** (réduction de 85 % à 95 %).
• **Traitement Vidéo :** Limitation stricte des lots à **20 fichiers vidéo maximum** (et non 50) afin de garantir la stabilité mémoire et CPU lors du transcodage natif.

01. Contexte et Objectifs du Projet

Ce cahier des charges définit le périmètre fonctionnel, l'architecture logicielle et le pipeline d'industrialisation d'une application de bureau multiplateforme dédiée à l'optimisation et la compression par lots d'images lourds (~15 Mo → 800 Ko à 2 Mo) et de vidéos (lots de 20 fichiers max). Basée sur **Tauri**, **React.js** et **Rust**, elle garantit des performances natives maximales et des installateurs légers pour Windows, Linux et macOS.

02. Expression des Besoins — Backlog des User Stories

Le tableau suivant décompose les besoins de traitement d'images et de vidéos en récits utilisateurs, critères d'acceptation et composants techniques :

ID & PRIO.	RÉCIT UTILISATEUR (USER STORY)	CRITÈRES D'ACCEPTATION (DOD)	IMPLÉMENTATION (REACT / RUST)
US-01 Haute (P0)	**En tant qu'**utilisateur, **Je veux** importer des images d'une taille moyenne de **15 Mo** (jusqu'à 50 fichiers), **Afin d'**obtenir des images compressées pesant entre **800 Ko et 2 Mo** en sortie.	Support JPG/PNG/WebP ; boucle d'ajustement qualité garantissant un poids final entre 800 Ko et 2 Mo sans dégradation visuelle notable.	Dropzone React + Crates Rust `image` / `mozjpeg` / `webp` parallélisés via `rayon`.
US-02 Haute (P0)	**En tant qu'**utilisateur, **Je veux** importer un lot limité à **20 fichiers vidéo maximum** (et non 50), **Afin de** compresser mes vidéos sans saturer la RAM ni le processeur.	Contrôle strict bloquant l'ajout au-delà de 20 vidéos avec message d'alerte UI ; file d'attente séquentielle et barre de progression temps réel.	Validation React + Sidecar Rust `ffmpeg` asynchrone (`tokio::process`) + événements IPC.
US-03 Moyenne (P1)	**En tant qu'**utilisateur, **Je veux** choisir le répertoire d'export et visualiser le ratio de compression avant/après, **Afin de** vérifier que les images respectent la cible 800 Ko – 2 Mo.	Sélection native du dossier de sortie ; rapport affichant la taille initiale (~15 Mo), la taille finale (800 Ko – 2 Mo) et le gain en %.	Plugin `@tauri-apps/plugin-dialog` + écriture non-destructive via `std::fs` en Rust.

03. Architecture Technique Tauri (React.js ↔ Rust)

L'application adopte le modèle multi-processus sécurisé de Tauri. Contrairement à Electron qui embarque Chromium et Node.js, Tauri utilise la WebView native du système d'exploitation pour le rendu React.js et délègue le décodage/encodage d'images lourdes (~15 Mo) et le transcodage vidéo au binaire Rust compilé.



Figure 1 — Flux d'exécution IPC entre la WebView React.js, le moteur de compression Rust et les systèmes cibles.

04. Découpage Modulaire & Algorithmes de Compression

- **Couche Présentation (`/src` — React.js + TypeScript) :** Zone de glisser-déposer validant les quotas en amont (maximum **50 images** de ~15 Mo ou **20 vidéos** par lot) et affichage en temps réel de la jauge de progression par fichier via les événements Tauri (`listen('compression-progress')`).
- **Moteur de Compression d'Images (`/src-tauri/src/image_engine.rs`) :** Traitement multi-threadé avec `rayon`. Pour ramener une image de **15 Mo** dans la fourchette stricte de **800 Ko à 2 Mo**, l'algorithme applique un redimensionnement adaptatif et une recherche dichotomique sur le facteur de qualité (Q=65..92).
- **Moteur Vidéo Limité à 20 Fichiers (`/src-tauri/src/video_engine.rs`) :** Vérification côté Rust du plafond `MAX_VIDEOS = 20`, puis exécution asynchrone d'un binaire `ffmpeg` embarqué (sidecar Tauri) avec encodage CRF 23–28 pour éviter toute saturation thermique ou mémoire.
- **Passage par Chemins Système :** Les fichiers de 15 Mo ou vidéos ne transitent jamais en base64 dans le pont IPC JSON ; seuls leurs chemins absolus (`PathBuf`) sont transmis de React vers Rust.

```
// Contrat IPC Rust — Vérification des quotas (20 vidéos max) & Cible Image (800 Ko - 2 Mo) const MAX_VIDEOS: usize = 20; const TARGET_MIN_BYTES: usize = 800 * 1024; const TARGET_MAX_BYTES: usize = 2 * 1024 * 1024; #[tauri::command] async fn process_media_batch(images: Vec<String>, videos: Vec<String>) -> Result<BatchReport, String> { if videos.len() > MAX_VIDEOS { return Err("Limite dépassée : 20 vidéos maximum par lot.".into()); } compress_all(images, videos, TARGET_MIN_BYTES, TARGET_MAX_BYTES).await.map_err(|e| e.to_string()) }
```

05. Stratégie de Compilation & Matrice des Cibles Multi-OS

La compilation croisée complète d'interfaces système natives (WebView2, GTK, Cocoa) nécessitant les bibliothèques dynamiques propres à chaque OS, l'industrialisation repose sur une matrice de runners natifs **GitHub Actions** pilotée par l'action officielle `tauri-apps/tauri-action`. Chaque publication de tag génère automatiquement les quatre formats d'installateurs exigés.

SYSTÈME CIBLE	EXTENSION	OUTIL DE PACKAGING	RUNNER GITHUB ACTIONS	CHEMIN DE L'ARTEFACT GÉNÉRÉ
Windows 10/11 (Standard)	`.exe`	NSIS (Nullsoft Scriptable Install System)	`windows-latest` (x86_64-pc-windows-msvc)	`bundle/nsis/*_x64-setup.exe`
Windows 10/11 (Entreprise GPO)	`.msi`	WiX Toolset v3 (Windows Installer)	`windows-latest` (x86_64-pc-windows-msvc)	`bundle/msi/*_x64_fr-FR.msi`
Linux (Debian / Ubuntu)	`.deb`	Tauri Debian Packager (dpkg)	`ubuntu-22.04` (x86_64-unknown-linux-gnu)	`bundle/deb/*_amd64.deb`
macOS (Apple Silicon / Intel)	`.dmg`	Apple Disk Image Bundler	`macos-latest` (aarch64 & x86_64)	`bundle/dmg/*_universal.dmg`

06. Prérequis Système & Configuration du Bundler

- **Environnement Windows (`.exe` & `.msi`) :** Le runner `windows-latest` intègre nativement MSVC (Visual Studio C++ Build Tools), WiX Toolset et NSIS. L'installateur vérifie la présence de *Microsoft Edge WebView2 Runtime* et l'installe automatiquement en mode silencieux (bootstrapper) s'il est absent du poste client.
- **Environnement Linux Debian/Ubuntu (`.deb`) :** Le job CI installe les en-têtes système indispensables avant la compilation Rust : `libwebkit2gtk-4.1-dev`, `build-essential`, `curl`, `wget`, `file`, `libssl-dev`, `libayatana-appindicator3-dev` et `librsvg2-dev`. Le paquet `.deb` généré déclare automatiquement ces dépendances pour `apt`.
- **Environnement macOS (`.dmg`) :** Le runner `macos-latest` compile à la fois pour l'architecture Apple Silicon (`aarch64-apple-darwin`) et Intel (`x86_64-apple-darwin`) afin de produire une image disque `.dmg` prête à être glissée dans le dossier `/Applications`.

> **Configuration déclarative dans `src-tauri/tauri.conf.json`**
> Pour activer simultanément les 4 formats d'installateurs lors de l'exécution du pipeline, la clé `bundle.targets` du fichier `tauri.conf.json` doit être configurée sur : `"targets": ["nsis", "msi", "deb", "dmg"]` avec un identifiant unique `"identifier": "com.organisation.projet"`.

07. Pipeline CI/CD GitHub Actions (`.github/workflows/release.yml`)

Ce workflow automatise la compilation du frontend React.js, du binaire Rust et la publication des installateurs `.exe`, `.msi`, `.deb` et `.dmg` dans une GitHub Release à chaque création d'un tag de version (ex. `git tag v1.0.0 && git push --tags`) :

```
name: Build & Release Tauri Multi-OS (.exe, .msi, .deb, .dmg) on: push: tags: [ 'v*' ]
workflow_dispatch: permissions: contents: write jobs: build-tauri: strategy: fail-fast:
false matrix: include: - platform: windows-latest # Génère les installateurs .exe (NSIS) et
.msi (WiX) args: '--bundles nsis,msi' - platform: ubuntu-22.04 # Génère le paquet
Debian/Ubuntu .deb args: '--bundles deb' - platform: macos-latest # Génère l'image disque
Apple .dmg args: '--bundles dmg' runs-on: ${{ matrix.platform }} steps: - uses:
actions/checkout@v4 - name: Dépendances système Linux (WebKitGTK & AppIndicator) if:
matrix.platform == 'ubuntu-22.04' run: | sudo apt-get update sudo apt-get install -y
libwebkit2gtk-4.1-dev libappindicator3-dev librsvg2-dev patchelf - uses: actions/setup-
node@v4 with: { node-version: 20, cache: 'npm' } - uses: dtolnay/rust-toolchain@stable -
uses: Swatinem/rust-cache@v2 with: { workspaces: './src-tauri -> target' } - run: npm ci -
uses: tauri-apps/tauri-action@v0 env: GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }} with:
tagName: v__VERSION__ releaseName: 'Release v__VERSION__' args: ${{ matrix.args }}
```

08. Plan de Recette & Validation des Livrables

PHASE DE CONTRÔLE	VÉRIFICATIONS TECHNIQUES & FONCTIONNELLES	CRITÈRE D'ACCEPTATION
Tests Unitaires & Qualité	Exécution de `cargo test` & `cargo clippy` côté Rust ; `vitest` & `eslint` côté React.js.	0 erreur de lint ; 100 % des tests critiques passants.
Recette Métier (Images & Vidéos)	Test d'un lot d'images de **15 Mo** en moyenne et tentative d'import de plus de **20 vidéos**.	100 % des images sorties entre **800 Ko et 2 Mo** ; rejet immédiat de la 21e vidéo.
Validation Packaging	Installation propre et lancement sur machines vierges Windows 11 (`.exe` / `.msi`), Ubuntu (`.deb`) et macOS (`.dmg`).	4 binaires publiés sur GitHub Releases et fonctionnels.

Validation Maîtrise d'Ouvrage (MOA) :
Responsable : [Nom du Product Owner / Client]
Date et signature : [JJ/MM/AAAA]

Validation Maîtrise d'Œuvre (MOE) :
Lead Tech Tauri : [Nom du Développeur Rust/React]
Date et signature : [JJ/MM/AAAA]